

Deploying an L1 on Avalanche

Workshop · June 28, 2026

Team1

Who Am I

Pedro Coelho do Nascimento

Rust Back-end Engineer · Full-Stack & Blockchain Developer · São Paulo, Brazil

- Rust engineer at **Bipa**: payment infrastructure, Central Bank compliance, AML
- Web3 open-source contributor: **Starknet**, **Stellar** ecosystems (OnlyDust)
- Developer at **Web3EduBrasil**: on-chain certificate issuance platform

Representing **Team1**

What We'll Cover

1. Avalanche architecture overview
2. The new L1 model (ACP-77)
3. Hands-on: deploying an L1
4. What's next?

Why Build an Avalanche L1?

Product and technical benefits

Your own economy

- Custom gas token: users pay in your token, not AVAX
- Custom fee model and gas parameters
- Fee revenue stays in your ecosystem

Sovereign by design

- Your validator set: permissioned or permissionless
- Compliance-friendly: no C-Chain exposure
- Isolated execution: other chains cannot affect your throughput

EVM and beyond

- Full EVM compatibility: port existing contracts
- Precompiles for minting, access control, fees
- ICM for native cross-chain messaging

Who Is It For?

Common use cases

Gaming

Fast finality, gas-free UX for players, custom reward token.

DeFi

Isolated liquidity, predictable costs, ~1-2s settlement.

Enterprise

Permissioned validators, private chain, regulatory compliance.

Payments

High throughput, stable fees, custom settlement token.

Section 1

Avalanche Architecture

The Primary Network

The **Primary Network** is the foundational layer — every validator on Avalanche must validate it.

P-Chain

Platform Chain

Coordinates validators, tracks active L1s, manages staking.

C-Chain

Contract Chain

EVM-compatible. Home of DeFi, smart contracts, AVAX token.

X-Chain

Exchange Chain

UTXO-based. Optimized for fast asset creation and transfers.

Snowman Consensus

Sub-sampled voting for fast finality

- Each validator repeatedly samples a small random subset of peers
- Switches preference when an α_p -majority agrees on a different option
- Increments confidence counter when an α_c -majority confirms the same option
- Decision finalized after β consecutive confirming rounds

Safety guarantee

$$P < \left(1 - \frac{\alpha_c}{k}\right)^\beta \approx 10^{-12}$$

Liveness requires >75% honest stake.

Symbol	Name	Default
k	Sample size (peers queried per round)	20
α_p	AlphaPreference (votes to switch preference)	15
α_c	AlphaConfidence (votes to increment counter)	15
β	Decision threshold (consecutive rounds)	20

Avalanche L1s

Independent blockchains with custom rules

Definition

An Avalanche L1 is an independent blockchain with its own validator set, tokenomics, and execution layer. Validators come to consensus and validate only that blockchain.

Each L1 specifies its own:

- Execution logic (VM)
- Fee structure
- State and networking
- Security model

Network-level benefits:

- Independent execution threads
- Higher aggregate throughput
- Isolated state and storage
- Cross-chain via ICM

Section 2

ACP-77: Reinventing Subnets

The Old Model: Subnets

Before Avalanche9000

- Subnet validators **required** to also validate the Primary Network
- Mandatory **2,000 AVAX** stake per validator on Primary Network
- At \$20/AVAX that's **\$40,000 locked per validator**
- Minimum viable Subnet needed 5 validators = \$200,000
- Validator set managed directly on the P-Chain by the Subnet owner

Bottleneck

Every Subnet validator had to run and sync the full Primary Network, increasing hardware costs and coupling performance to Primary Network load.

Avalanche9000: The Etna Upgrade

The largest network upgrade in Avalanche's history

Economic barriers removed

2,000 AVAX stake requirement **eliminated**. L1 validators pay only a continuous P-Chain fee of **1-10 AVAX/month** per validator.

Improved compliance

Validator sets are fully separate from the Primary Network. Institutions can launch private L1s with **zero exposure** to public permissionless chains.

Running costs reduced

Validators only run the L1, no need to sync the Primary Network. Lower hardware requirements, simpler ops.

Fault isolation

Congestion or outages on the Primary Network no longer affect L1 performance, and vice versa.

Implemented via ACP-77 (Reinventing Subnets), supersedes ACP-13

ACP-77: What Changed

Activated · Standards Track

1. Separation

Subnet validators no longer required to validate the Primary Network. 2,000 AVAX requirement removed.

2. Sovereignty

Validator set management moves from P-Chain to a **Validator Manager contract** on the L1 itself.

3. Continuous fees

L1 validators pay a continuous fee to the P-Chain — no large upfront stake needed.

ACP-77 supersedes ACP-13 (Subnet-Only Validators)

Subnet vs. L1: Side by Side

	Subnet (legacy)	L1 (ACP-77)
Validator set	Must validate Primary Network	Sovereign, custom set
Stake required	2,000 AVAX minimum	No minimum (continuous fee)
Set management	P-Chain / subnet owner	Validator Manager contract
Primary net sync	Full sync required	Header-only (partial sync)
Finality	1–2 s	1–2 s
Interoperability	ICM	ICM

Section 3

Deploying Your L1

What We're Building

Four-step process



Prerequisites:

- Core Wallet (set to Testnet)
- Testnet AVAX on P-Chain
- AWS EC2 instance (we'll set it up together)

Tools:

- [Avalanche Builder Console](#)
- Docker on EC2 for AvalancheGo
- MetaMask / Core Wallet for RPC testing

Node Roles

Validator · RPC · Combined

Validator node

Port 9651 (P2P)

Participates in consensus and produces blocks. RPC bound to localhost only. Not exposed publicly.

RPC node

Port 9650 (HTTP)

Serves JSON-RPC to wallets and dApps. Not part of the validator set. No consensus role.

Validator + RPC

Ports 9650 + 9651

Both roles on one machine. Simpler ops, higher resource use. **What we use today.**

Workshop topology

Single EC2 running AvalancheGo in **validator-rpc** mode. **9651**: open inbound/outbound for P2P consensus. **9650**: bound to localhost; a reverse proxy (nginx / Caddy) on **443/80** terminates TLS and forwards to it. Adequate for testnet. Production separates validator and RPC nodes.

Section 4

What's Next?

Customizing Your L1

EVM precompiles — Solidity contracts at known addresses

- **NativeMinter** — mint your gas token on-chain
- **FeeManager** — adjust fee config without a hard fork
- **RewardManager** — configure block reward distribution
- **ContractDeployerAllowList** — restrict who can deploy contracts
- **TxAllowList** — restrict who can send transactions
- **WarpMessenger** — send cross-chain messages via ICM

Enable in genesis

Precompiles are activated in `genesis.json` under the `config` block. Access control lists define admin and enabled addresses.

Going Further

Natural next steps from what we built today

PoA → PoS

Replace the PoA manager with a **Staking Manager**. Anyone who stakes your native token (or an ERC-20) can join the validator set, no owner approval needed.

Multisig governance

Wrap the PoA manager in a **Safe multisig** so no single key controls validator adds/removes. Critical before mainnet.

ICM: Interchain Messaging

Send arbitrary messages between Avalanche L1s. Token bridges, cross-chain app logic, shared state. Needs a **Warp relayer** running alongside your node.

Beyond EVM

HyperSDK: build a custom VM in Go with full control over state, execution, and fee models. For when the EVM is the bottleneck, not the answer.

Resources

Documentation & tooling:

- [build.avax.network](#) — Builder Hub
- [L1 Toolbox](#)
- [Academy courses](#)

Key ACPs:

- ACP-77: Reinventing Subnets
- ACP-13: Subnet-Only Validators (superseded)

Community:

- [discord.gg/avax](#)
- [github.com/ava-labs](#)

Today's code:

- [ava-labs/avalanchego](#)
- [ava-labs/subnet-evm](#)
- [ava-labs/icm-contracts](#)

What We Built

From zero to a live Avalanche L1 on Fuji

Your L1: live on Fuji

- Custom subnet & blockchain registered on P-Chain
- AvalancheGo validator node running via Docker
- Validator Manager contract (PoA) deployed
- RPC endpoint testable with MetaMask / Core

Key concepts covered

- Primary Network: P/C/X-Chain separation
- ACP-77: sovereign validator sets, no 2,000 AVAX requirement
- Validator Manager: on-chain governance
- Node roles: validator, RPC, combined topology

Questions?

Team1